

6 – SEQUENCES

A sequence is an ordered collection of items that supports operations such as indexing, slicing, iteration and membership testing. They form a core part of the python language with many built-in types as sequences.

Sequences may be categorized into two main groups: Immutable and mutable.

Immutableles

Immutable types include `str`, `tuple` and `bytes`. Variables of these types do not change after initialization. When a variable of these type is changed, the new value is assigned to a new location. The existing variable reference is changed to point to the new location.

Information

The numeric types `int` and `float` are immutable as well as the set type `frozenset`.

```
# Demonstrates immutability
a = 'hello'
b = a
print(f'{a:<20} {b}')
hello                hello

print(f'{id(a):<20} {id(b)}')
1389813655632        1389813655632

a = 'world'           # a new value is assigned to a
print(f'{a:<20} {b}')
world                hello

print(f'{id(a):<20} {id(b)}')
1389818780176        1389813655632
```

Mutable

Mutable types include `list` and `bytearray`, and most custom type (types that are user defined).

When a variable of these types is changed, the new value is assigned to the existing location.

Information

The mapping type `dict` is immutable as well as the set type `set`.

```
# Demonstrates mutability
a = [1]
b = a
print(f'{a} {b}')
[1] [1]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992

a.append(2)
print(f'{a} {b}')
[1, 2] [1, 2]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992
```

6.a – String

A string is a sequence of characters that is delimited with either a pair of single quotes, double quotes or triple single or triple double quotes. In a string literal, you may not use mis-match quotes as your delimiter. i.e. the start and end delimiters must be identical in a literal definition.

Strange fact

Python does not have a native `char` data type as does most modern languages.

```
a = 'Hello '
```

```
b = "world"
c = '''This string
can span multiple
lines.'''
```

6.a.1 – String Creation Methods

The `str()` function can create a string from pretty much any literal. There are also lots of functions that returns a string.

```
>>> str('Hello World')
'Hello World'

>>> str(3.14159)
'3.14159'

>>> str([1, 2])
'[1, 2]'

>>> str(True)
'True'

>>> str(None)
'None'
```

6.a.1 – String Concatenation

You can add strings to string to create a new string. This is called concatenation and is supported by most modern high-level language. It results in a new string. In python you are able to multiple a string by an integer.

Fun fact

Ruby and PERL also allow you to multiple a string by an integer.

```
>>> 'Hello ' + 'Narendra'
'Hello Narendra'

>>> 'Hi ' * 3
```

```
'Hi Hi Hi '
```

6.a.2 – Escaping a String

Because quotations are special, (it denotes the start and end of a string) if you want to use a quote in a string, you need to escape it with a `\`

```
>>> 'She said \'hi\''
"She said 'hi'"

>>> print('Hi\nNarendra')           #embedding a newline character
Hi
Narendra

>>> print('Hi\tNarendra')           #embedding a tab
Hi      Narendra
```

6.a.3 – Raw strings

All string in python are checked for meta-characters such as `\n` (newline) and `\t` (tab). As you may notice, these sequences are prefixed with the backslash character (`\`). If you want to use the backslash as part of your string, then you have to prefix it with another backslash. Thus, `print('\\/n')` with output `'/n'`. This can be tedious to type and hard to read, so the language designers came up with a technique that allows a programmer to indicate to the interpreter that a string should not be processed. Raw strings are especially convenient when using regular expressions.

Fun fact

Other languages that support raw strings are C#, C++ and Rust. Java does not support raw strings.

A prefix of `r` (or `R`) before a string tells python that the string should not be processed.

```
>>> print(r'\n')                     #raw string
\n

>>> print('\n')                     #normal string

>>> print('\\n')                     #normal string
```

```
\n
>>> print(r'\n')           #raw string
\\n
>>>
```

6.a.4 – Combining F-Strings and raw strings

It is possible to embed an expression within a raw string as the example below shows:

```
>>> base = 'facultyweb.centennialcollege.ca'
>>> print(rf'http://{base}/npersh')
http://facultyweb.centennialcollege.ca/npersh
```

6.b – List

A list is a collection of values that can be of any type. It is mutable i.e., it is possible to modify an existing list. You may perform indexing and slicing on a list.

6.b.1 – Creating Lists

You may create list using the following statements:

```
a = list()
b = []
c = [9, 8, 7]
d = 'Mary had a little lamb'.split()
e = [0, 'one', [2], 3.0, True]
```

6.b.2 – Adding Lists Together

You can extend a list with another list by using the extend keyword, or the + symbol.

```
list1 = [1, 2, 3]
list2 = [9, 8, 7]
new_list = list1 + list2
list1 += list2
```

6.b.3 – Adding an Item to a List

If you just want to add a single item to a list, you need to use the `append()` method.

```
fruits = ['apple', 'banana', 'orange']
fruits.append('pear')
```

The in operator

This operator returns True if the item exists in the list.

```
colors = ['red', 'green', 'blue']
color = 'white'
print(f'{color in colors}')      #display False
```

List Indexing

To get hold of a particular item from a list you can use its index number. This number can also be negative, if you want to start counting from the end of the list.

```
letters = ['a', 'b', 'c']
letters[0]          #give 'a'
letters[-1]         #the last item 'c'
letters[0] = 'z'    #sets the first item to 'z'
```

List Slicing

Using the list index and the colon symbol you can slice up a list to get only the portion you want. Start is included, but end is not.

```
#list[start:end:step]
letters = ['a', 'b', 'c', 'd']
letters[1:3]      #Result: ['b', 'c']
```

6.c – Common Sequence Operations

The following operations can be performed on all sequences.

6.b.1 – min(), max() sum()

These work only if all the items are numeric.

```
>>> max((8, 6, 4.6, -9))
8
>>> sum([8, 6, 4.6, -9])
9.600000000000001
>>> min((8, 6, 4.6, -9))
-9
```

6.b.1 – Length

Returns the count of the number of first-level items in the sequence.

```
>>> len('Narendra')
8
>>> len((8, 6, 4.6, -9))
4
>>> len([8, [], 'hello'])
3
```

6.b.1 – Membership test

The **in** operator is used to determine if an item is present in a list.

```
>>> print('N' in 'Narendra')
True
>>> print('E' in 'Narendra')
False
>>> print([] in [8, [], 'hello'])
```

```
True
>>> print(0 in [8, [], 'hello'])
False
```

6.b.1 – Concatenation

Concatenation or joining is accomplished with the plus operator. This is only supported between two identical types: str and str, tuples and tuples and list and list

```
>>> 'Narendra ' + 'Pershad'    # two strings
'Narendra Pershad'
>>> (1, 2) + (3, 4)             # two tuples
(1, 2, 3, 4)
>>> [1, 2] + [3, 4]             # two lists
[1, 2, 3, 4]
```

For set use the union (|) operator

```
>>> {1, 2} | {3, 4}
{1, 2, 3, 4}
```

For dict use the | operator

```
>>> {1: 'one'} | {2: 'two'}
{1: 'one', 2: 'two'}
```

6.b.1 – Repetition

```
>>> 'Narendra ' * 3           #string
'Narendra Narendra Narendra '
>>> (1, 2) * 2                 #tuple
(1, 2, 1, 2)
>>> [1, 2] * 2                 #list
[1, 2, 1, 2]
```

6.b.1 – Iteration

6.b.1 – Indexing

6.b.1 – Slicing

6.d – Summary

This chapter introduced **sequences**, ordered collections that support indexing, slicing, iteration, and membership testing. Python's built-in sequence types fall into two categories: **immutable** (str, tuple, bytes) and **mutable** (list, bytearray). The chapter focused primarily on strings and lists, the two most commonly used sequence types.

Key takeaways:

- **Immutability vs. mutability:**
 - Immutable objects (`str`, `tuple`, `bytes`) cannot be changed after creation. Assigning a "new value" rebinds the name to a new object.
 - Mutable objects (`list`, `bytearray`, `set`, `dict`) can be modified in place; changes are visible through all references to the same object.
- **Strings:**
 - Created with single, double, or triple quotes; triple quotes allow multi-line strings.
 - Concatenation with `+`; repetition with `*`.
 - Escape sequences: `\n` (newline), `\t` (tab), `\'` (quote), `\\` (backslash).
 - Raw strings (`r' . . . '`) disable escape processing — useful for regex and Windows paths.
 - F-strings can be combined with raw strings (`rf'...'.`).
- **Lists:**
 - Created with `list()`, `[]`, or `str.split()`.
 - Combined with `+` (new list) or `extend()` / `+=` (in place).
 - Single items added with `append()`.
 - Support indexing (including negative indices) and slicing (`list[start:end:step]`, end-exclusive).
- **Common sequence operations (work on all sequences):**
 - `min()`, `max()` — work on comparable items; `sum()` — numeric only.
 - `len()` — counts first-level items.
 - `in` — membership test.
 - `+` — concatenation (same type only).
 - `*` — repetition.
 - Iteration with `for`.
 - Indexing and slicing.
- **Merging collections:** sets and dicts use `|` for union/merge (not concatenation).
- **Note:** `tuple`, `bytes`, and `bytearray` were mentioned but not covered in depth; strings and lists were the primary focus.

In short: sequences are the workhorses of Python data handling. Understanding immutability, indexing, slicing, and the difference between in-place vs. new-object operations is essential for writing correct and efficient code.

6.e – Non coding exercises

1. What does `'Hi' * 3` produce?
2. What does `letters[1:3]` return for `letters = ['a','b','c','d']`?
3. What does `len([8, [], 'hello'])` return?
4. Which operator merges two dictionaries?
5. What does `r'\n'` represent?

6. Strings are mutable in Python. (True or false?)
7. `min()` and `max()` work only on numeric types. (True or false?)
8. Slicing includes the end index. (True or false?)
9. `+=` on a list modifies the list in place. (True or false?)
10. `id()` values are the same across all Python runs. (True or false?)
11. A raw string disables escape sequence processing.
12. `bytes` is a mutable sequence type. (True or false?)
13. `in` works on strings, lists, and tuples. (True or false?)
14. What is the result of `'Hello ' + 'World'` ?
15. What is the result of `'ab' * 2` ?
16. What is the result of `(1, 2) + (3,)`?
17. What is the result of `[1, 2] * 2` ?
18. What is the result of `len('Narendra')`?
19. What is the result of `'N' in 'Narendra'`?
20. What is the result of `[] in [8, [], 'hello']` ?
21. What is the result of `0 in [8, [], 'hello']`?
22. What is the result of `max((8, 6, 4.6, -9))`?
23. What is the result of `min(['apple', 'zebra', 'mango'])`?
24. What is the result of `{1, 2} | {3, 4}`?
25. What is the result of `{1: 'one'} | {2: 'two'}`?
26. What is the result of `letters = ['a','b','c','d']; letters[-1]`?
27. What is the result of `letters[0:4:2]`?
28. What is the difference between immutable and mutable types. Give one example of each.
29. Why does `id()` change after assigning a new value to a string variable but not after `append()` to a list?
30. What is the difference between `+` and `+=` on a list?
31. Why are raw strings useful? Give a practical example.
32. What is the difference between `str()`, `repr()`, and `print()`?
33. Explain slicing with a step. What does `[:2]` do?
34. Why does `'Hi' * 3` work but `'Hi' * 'x'` does not?
35. What is the difference between `in` for a string and `in` for a list?